IFCD107 · ESPECIALISTA EN INTELIGENCIA ARTIFICIAL

Árboles en cadena que corrigen errores

Boosting: el paso (learning_rate), la parada temprana, XGBoost y LightGBM

Sprint 8 · S08_03 · cuaderno: 25 min en clase (ejercicios 1 y 2)

LA IDEA

Cada árbol repasa los fallos del anterior

Como los simulacros de un examen: tras cada uno repasas **solo lo que has fallado**. Ejemplo con 6 reservas y 3 árboles de una sola pregunta (predecimos el precio por noche):

reserva	suite (1 = sí)	alta (1 = sí)	real (€)	media	tras árbol 1	tras árbol 2	tras árbol 3
A	0	0	60	100	87,5	81,25	76,25
B	0	1	80	100	87,5	81,25	86,25
C	1	0	140	100	125	137,5	132,5
D	1	1	160	100	125	137,5	142,5
E	0	0	70	100	87,5	81,25	76,25
F	0	1	90	100	87,5	81,25	86,25
error medio (€)				33,33	17,50	11,25	9,58

Árbol 1: mira **suite**, suma +50 a las suites y −25 al resto, pero con **learning_rate=0,5** aplica **la mitad**.

BOSQUE FRENTE A BOOSTING · Y EL PASO

Dos formas de juntar árboles

	Bosque (bagging)	Boosting
Árboles	Grandes, independientes, en paralelo	Pequeños, en cadena: cada uno corrige al anterior
Reduce sobre todo	Varianza (el modelo cambia mucho si cambian los datos)	Sesgo (el modelo es demasiado simple y falla siempre en lo mismo)
Riesgo	Poco sobreajuste	**Sobreajusta si se añaden demasiados árboles**

learning_rate	error medio tras 3 árboles
0,2	17,80
0,5	9,58
1,0	3,33

El paso: pequeño = más árboles

Un paso pequeño avanza más despacio y necesita **más árboles**, pero afina más. Con datos reales y ruido, un paso grande sobreajusta antes.

SOBREAJUSTE · EJERCICIO 1 · CUADERNO (CLASE)

Demasiados árboles memorizan

Árboles (paso 0,2)	AUC entrenamiento	AUC validación
1	0,698	0,692
12 · el mejor	0,772	0,741
50	0,819	0,737
100	0,854	0,722
300	0,935	0,702

```
mejor_n_arboles = int(np.argmax(auc_val_por_arbol)) + 1
caida_final = max(auc_val_por_arbol) - auc_val_por_arbol[-1]
```

Entrenamiento sube siempre; validación llega al máximo con 12 árboles y cae 0,039 si seguimos hasta 300.

PARADA TEMPRANA

Para cuando el simulacro deja de mejorar

3840
Entrenar (**Xfit**)

960
Validar: decide cuándo parar (**Xval**)

1200
Test: **no se toca**

Ronda	AUC validación	Mejor hasta ahora	Rondas sin mejorar
6	0,740	0,741	1
7	0,742	0,742	0
8	0,741	0,742	1
9	0,739	0,742	2
10	0,738	0,742	3 · **¡para!**

Ejemplo con paciencia de 3 rondas (la ronda 5 llegó a 0,741): se detiene y **se queda con la ronda 7**. En el cuaderno la paciencia es de 30 (**early_stopping_rounds=30**).

XGBOOST · EJERCICIO 2 · CUADERNO (CLASE)

Un máximo alto y que pare solo

Hiperparámetro	Qué controla
n_estimators	Máximo de árboles (con parada, se pone alto)
learning_rate	Paso de cada árbol (menor = más fino)
max_depth	Profundidad de cada árbol (2 a 6 suele bastar)
subsample · **colsample_bytree**	Fracción de filas / columnas por árbol
reg_lambda · **min_child_weight**	Regularización y tamaño mínimo de nodo
early_stopping_rounds	Rondas sin mejorar para detenerse

69

arboles_xgb (**best_iteration**)

0,745

auc_test_xgb

```
modelo_xgb = xgb.XGBClassifier(n_estimators=1000, learning_rate=0.05, max_depth=3,
        subsample=0.8, colsample_bytree=0.8, early_stopping_rounds=30, eval_metric="auc")
modelo_xgb.fit(Xfit, yfit, eval_set=[(Xval, yval)], verbose=False)
```

LIGHTGBM · EJERCICIO 3 (CASA)

Crece por hojas: manda num_leaves

XGBoost: por niveles

max_depth=3 permite como mucho 2^3 = **8 hojas**, nivel a nivel.

LightGBM: por hojas

num_leaves=8 fija las **8 hojas**; crece siempre por la hoja que más mejora. **min_child_samples** evita hojas diminutas.

```
modelo_lgb = lgb.LGBMClassifier(n_estimators=1000,
    learning_rate=0.05, num_leaves=8, subsample=0.8,
    subsample_freq=1, colsample_bytree=0.8, verbose=-1)
modelo_lgb.fit(Xfit, yfit, eval_set=[(Xval, yval)],
    eval_metric="auc",
    callbacks=[lgb.early_stopping(30, verbose=False)])
```

49

arboles_lgb

0,745

auc_test_lgb

QUÉ MIRA EL MODELO Y MINI-RETO (CASA)

Tres implementaciones, casi el mismo AUC

Implementación	AUC en test	Árboles	Tiempo de ajuste
sklearn_gb	0,744	139	≈ 0,8 s
xgboost	0,745	69	≈ 0,1 s
lightgbm	0,745	49	≈ 0,1 s

Qué mira XGBoost (ganancia)

Encabezan **deposito_sin_deposito** (0,083), **antelacion_dias** (0,082) y **canal_web_intermediaria** (0,080). Describen el **modelo**, no las causas.

Los tiempos cambian

Salen de nuestra prueba y varían entre ejecuciones y ordenadores. Para comparar, mismos datos y mismo **n_jobs**.

La implementación importa menos que los datos y la validación: decide el coste (tiempo, mantenimiento, explicabilidad).

HOY HAS APRENDIDO

Pequeños pasos, en cadena y con freno

Boosting

Árboles pequeños en cadena; cada uno corrige los errores del anterior

Paso y parada

learning_rate y nº de árboles se equilibran; la parada temprana usa validación, nunca el test

XGBoost y LightGBM

Parecen iguales en AUC; difieren en detalles (**max_depth** o **num_leaves**) y en coste

Siguiente: S08_04 · máquinas de vectores de soporte (SVM)